

Code-Layout, Readability and Reusability

Date 29 September 2026

Team ID SWTID-2026-7446

Project Name ComicCraft - AI Comic Story Creator using Gemini Models

Maximum Marks 5 Marks

Code-Layout, Readability and Reusability: This document evaluates the quality of the ComicCraft codebase in terms of its structure, readability, and reusability. The project uses a modular FastAPI backend, separate AI-generation modules, Jinja2 templates, layout building, and PDF export components.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Python/FastAPI modules follow a consistent structure.
2	Proper File Structure	Files and folders are logically organized	Yes	Separate modules for routes, Gemini models, image generation, layout, exporters, templates and static files.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as prompt, character_name, setting, tone, art_style and layout are descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as generate_outline(), generate_story(), generate_image(), build_comic_layout() and save_pdf() clearly indicate their purpose.
5	Code Comments	Inline and block comments explain logic	Partial	The documented project structure is clear; comments can be expanded where complex AI/API processing needs explanation.
6	Modular Design	Code is split into reusable functions/modules	Yes	AI generation, image creation, layout construction, routing and PDF export are separated into reusable modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	The workflow is divided into dedicated functions, reducing repeated processing logic.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	FastAPI HTTPException and try-except blocks are used for API and JSON-related failures.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	generate_outline()	Python + Gemini Flash	Comic panel outline generation	High
2	generate_story()	Python + Gemini Pro	Narration and character dialogue generation	High
3	generate_image()	Python + Stable Diffusion / Diffusers	Illustration generation for each panel and image testing	High
4	build_comic_layout()	Python	Combines panel images with corresponding story content	High
5	save_pdf()	Python + FPDF	Creates downloadable multi-page comic PDFs	High
6	routes.py	Python + FastAPI	Form-based and JSON-based comic generation workflows	Medium
7	Jinja2 templates	HTML + CSS + Jinja2	Input, comic preview and export-success pages	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Modular separation of FastAPI routing, AI models, image generation, layout construction, templates and PDF export.
Readability	4	Descriptive function/module names and a clear generation workflow make the code understandable.
Reusability	5	Core generation and export functions are independently reusable across form and JSON workflows.
Documentation / Comments	4	The project documentation clearly explains the workflow; additional inline comments could improve complex sections.
Overall Score	4.5 / 5	Strong modular structure and reusable components with room for additional code-level documentation.

ComicCraft – Project Reference

Team ID: SWTID-2026-7446

Project: ComicCraft - AI Comic Story Creator using Gemini Models

Item	Details
Application	Web-based AI application for generating personalized comic stories and illustrations from user prompts.
Backend	FastAPI
Story Generation	Gemini Flash for structured comic panel outlines and Gemini Pro for detailed narration and dialogue.
Image Generation	Stable Diffusion with Hugging Face Diffusers.
Frontend	HTML, CSS, and Jinja2 templates.
Export	FPDF-based multi-page PDF generation.
Core Modules	generate_outline(), generate_story(), generate_image(), build_comic_layout(), save_pdf(), and routes.py.